

User Acceptance Testing (UAT) Execution Document

Date	19 September 2026
Team ID	
Project Name	ShopEZ — Apparel E-Commerce Application
Maximum Marks	4 Marks

1. Project Overview & Testing Environment

Project Name: ShopEZ E-Commerce Application

Project Description: A full-stack MERN application featuring variant-level (size + color) stock tracking, JWT-based authentication with role-based access, a persistent cart, and a complete admin panel for products, orders, and site settings.

Project Version: v1.0.0

Testing Period: 12 September 2026 to 19 September 2026

Testing Scope:

- Customer registration, login, and JWT-protected route access.
- Product browsing, category/search filtering, and per-variant stock display.
- Cart management, checkout, and order placement with stock validation.
- Order status tracking, product reviews, and admin CRUD for products, orders, and site settings.

Testing Environment:

- **URL/Location:** Local development environment (http://localhost:5173 client, http://localhost:8000 API)
- **Credentials:** Customer: Sarah_test@example.com / Sarah@1234 | Administrator: arya.sakore24@gmail.com / 1234567890

2. Executed User Acceptance Test Cases

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
TC-001	User Registration	1. Go to /register. 2. Enter name, email, password. 3. Submit.	Account is created; password is hashed; JWT returned and user is logged in.	Account created successfully; redirected to homepage logged in.	Pass
TC-002	User Login	1. Go to /login. 2. Enter valid credentials. 3. Submit.	User is authenticated and receives a valid JWT.	Login succeeded; token stored and	Pass

				Navbar reflects logged-in state.	
TC-003	Category Filter & Search	1. Open /products. 2. Select a category filter. 3. Enter a search term.	Product grid updates to match filter/search without a full page reload.	Grid updated correctly for both category filter and search.	Pass
TC-004	Variant Stock Accuracy	1. Open a product detail page. 2. Select different size/color combinations.	Stock count shown updates live to reflect the exact selected variant.	Stock value changed correctly for each size/color combination tested.	Pass
TC-005	Add to Cart & Quantity Update	1. Add a variant to cart. 2. Open /cart. 3. Change quantity.	Cart total recalculates correctly; quantity change is persisted.	Cart total updated accurately after quantity change.	Pass
TC-006	Checkout & Order Placement	1. Proceed to checkout. 2. Enter address & payment method. 3. Place order.	Stock is decremented, an order is created, and cart is cleared.	Order created with correct total; cart emptied; stock reduced.	Pass
TC-007	Order Status Tracking	1. Log in as admin. 2. Update an order's status. 3. Log in as the customer and check Profile.	Updated status is reflected on the customer's order history.	Status change appeared correctly on the customer's profile page.	Pass
TC-008	Admin Product CRUD	1. Log in as admin. 2. Create a new product with variants. 3. Edit and then delete it.	Product is created, updated, and removed correctly in the catalog.	All three operations completed successfully and reflected instantly.	Pass

3. System Defect Register & Bug Tracking

These are defects genuinely encountered and resolved during development — not simulated — kept here as an honest record of the debugging process.

Bug ID	Bug Description	Steps to Reproduce	Severity	Status	Resolution
BG-001	Registering a new user crashed the server with “TypeError: next is not a function”.	1. POST /api/users/register. 2. Mongoose pre-save hook attempts to hash password.	High	Closed	Removed the legacy next() callback from the async pre-save hook — newer Mongoose versions don't support mixing async functions with a next parameter.
BG-002	Client dev server failed to start: “Failed to resolve import react-router-dom”.	1. Run npm install in the project root instead of client/. 2. Run npm run dev.	Medium	Closed	Re-ran the install command from inside the client folder so the dependency landed in the correct node_modules.
BG-003	Discount percentage was shown on product cards but not actually applied to the cart or order total.	1. Add a discounted product to cart. 2. Check cart total vs. displayed price.	High	Closed	Updated cart, checkout, and order-placement price calculations to apply discountPercent consistently with the displayed price.
BG-004	Backend server accidentally bound to port 5173 (the client's port) after an .env edit.	1. Edit server/.env. 2. Run npm run dev.	Low	Closed	Corrected the PORT value in .env back to 8000.

4. Formal Acceptance Sign-Off

Tester: Project developer, self-tested against all executed test cases above

Date of Test Completion: 19 September 2026

Notes:

- All core user flows (registration through order tracking) and all admin flows (product, order, and settings management) have been manually tested end to end.
- All identified defects have been resolved and re-verified.
- Known limitations: payment methods other than Cash on Delivery do not process real payment; the application is not yet deployed to a public URL.